



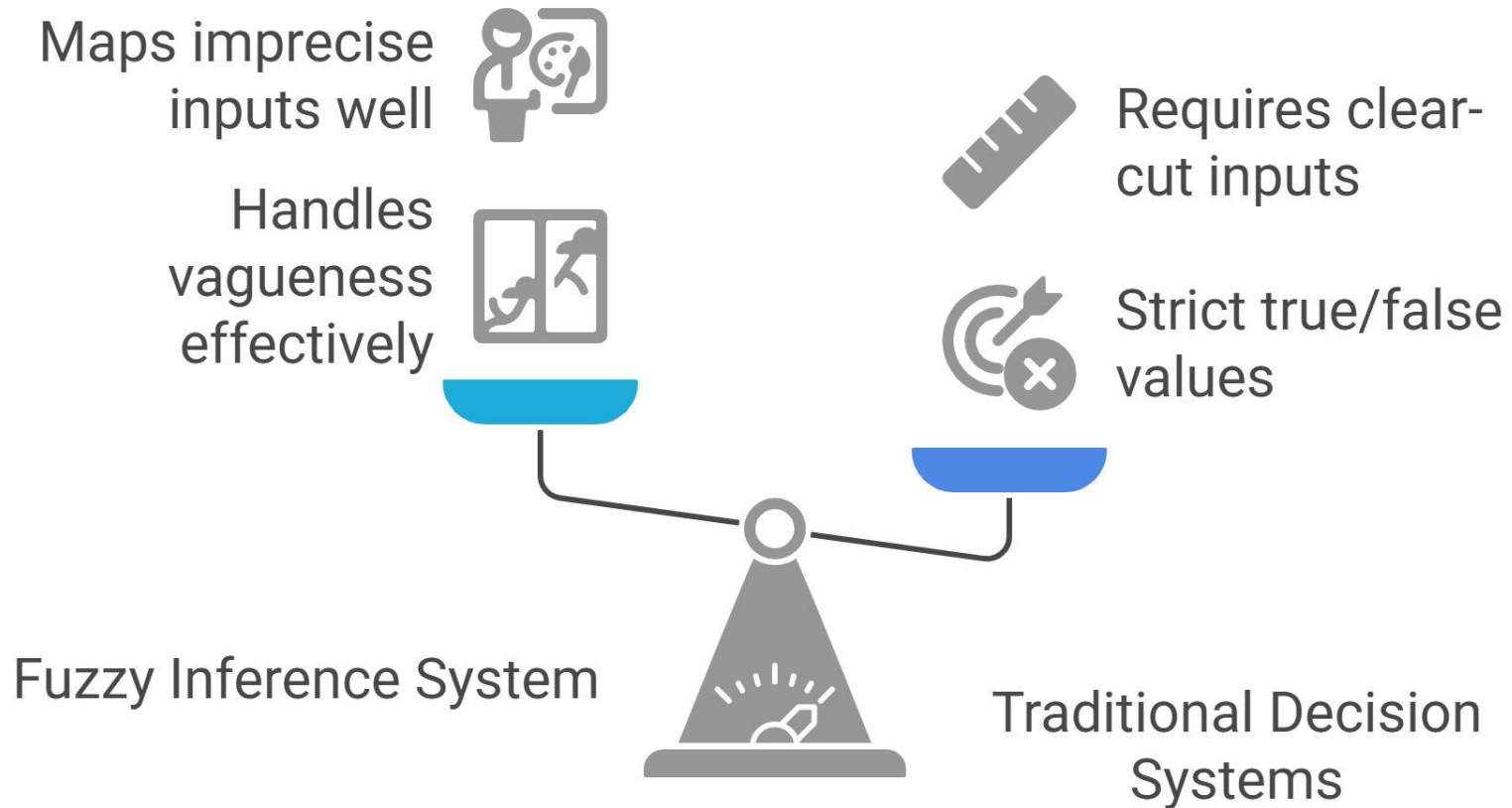
Fuzzy Logic & Neural Networks (CS-514)

Dr. Sudeep Sharma

IIIT Surat

sudeep.sharma@iiitsurat.ac.in

Fuzzy Inference System



Comparing Fuzzy and Traditional Decision Systems

Fuzzy Inference System

➤ Key Benefits of FIS:

- **Smooth Transitions:** FIS allows for gradual adjustments, avoiding abrupt on/off cycles.
- **Human-Like Reasoning:** FIS mimics human thinking by treating inputs as a spectrum rather than rigid categories.
- **Adaptability:** FIS responds flexibly to changing conditions.

Fuzzy Inference System

Example: AC System Comparison

Traditional Logic	Fuzzy Inference System
AC turns on when $>30^{\circ}\text{C}$ and off when $<30^{\circ}\text{C}$.	At 29°C , the system considers 0.7 “Warm” and 0.3 “Hot,” adjusting cooling gradually.
Leads to abrupt temperature changes and discomfort.	Results in smoother transitions and a more comfortable environment.

Fuzzy Inference System

- Fuzzy inference is the process of formulating the mapping from a given input to an output using fuzzy logic.
- The mapping then provides a basis from which decisions can be made.
- Fuzzy rules are a collection of linguistic statements that describe how the FIS should make a decision regarding classifying an input or controlling an output.
- The process of fuzzy inference involves all the pieces that are described in Membership Functions, Logical Operations, and If-Then Rules.

Fuzzy Inference System

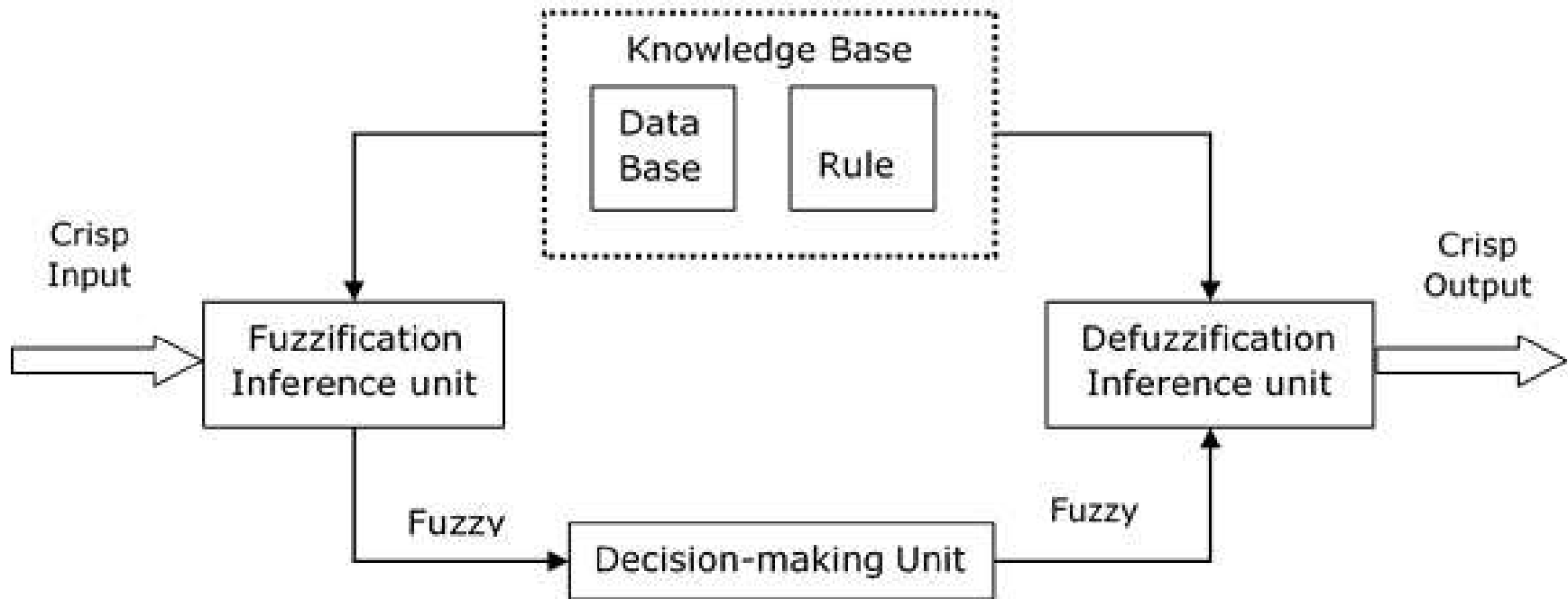


Fig: Functional Blocks of FIS

- **Rule Base** — It contains fuzzy IF-THEN rules.
- **Database** — It defines the membership functions of fuzzy sets used in fuzzy rules.
- **Decision-making Unit** — It performs operation on rules.
- **Fuzzification Interface Unit** — It converts the crisp quantities into fuzzy quantities.
- **Defuzzification Interface Unit** — It converts the fuzzy quantities into crisp quantities.

Fuzzy Inference System

To compute the output of this FIS given the inputs, one must go through six steps:

1. Define the Input, Output MFs & a set of fuzzy rules.
2. Fuzzifying the inputs using the input membership functions.
3. Combining the fuzzified inputs according to the fuzzy rules to establish a rule strength.
4. Finding the consequence of the rule by combining the rule strength and the output membership function.
5. Combining the consequences to get an output distribution, and
6. Defuzzifying the output distribution.

Fuzzy Inference System

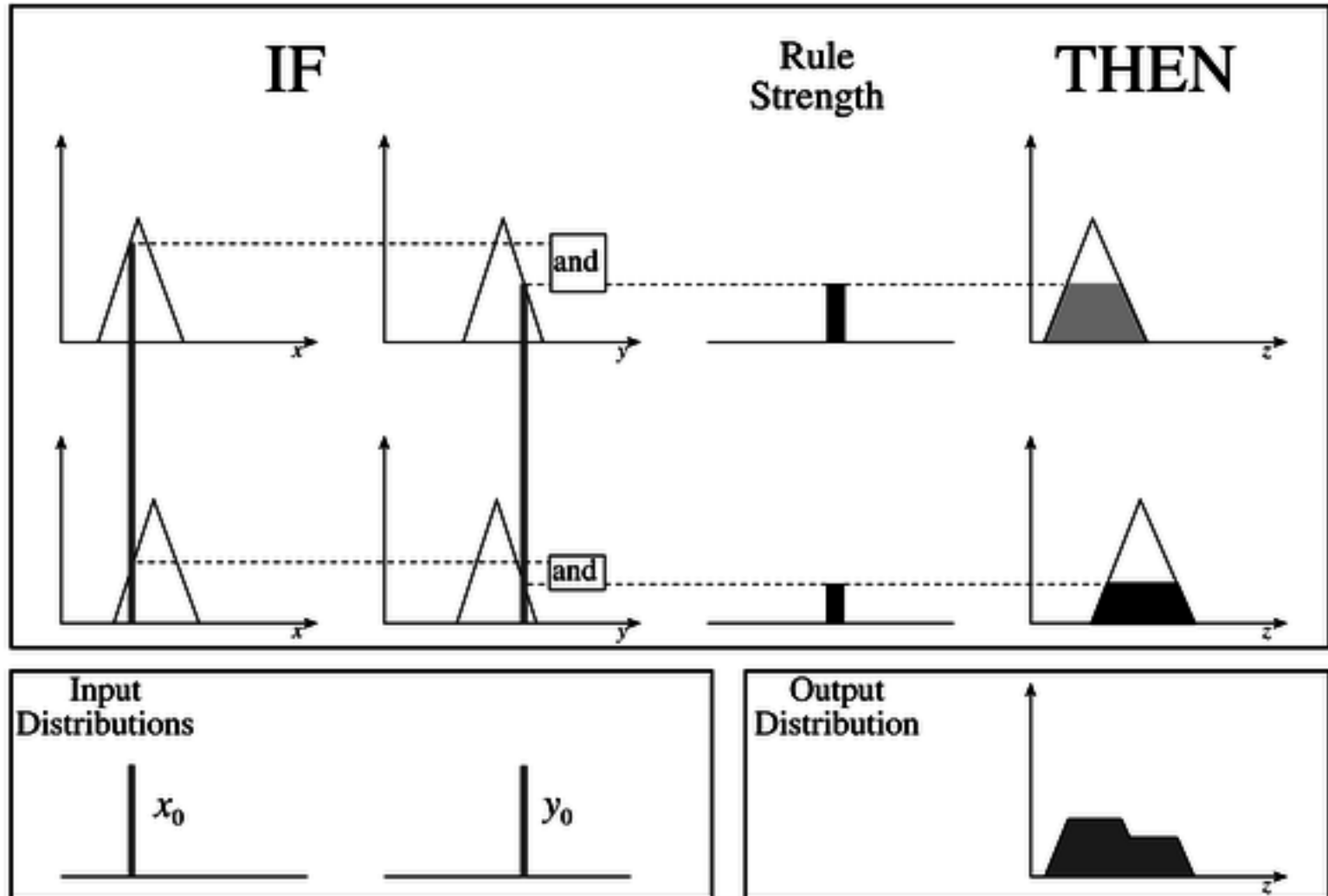
Popular Fuzzy Inference Systems (Fuzzy Models)

1. Mamdani Fuzzy Models
2. Sugeno Fuzzy Models

The *Mamdani FIS*, also known as the Max-Min method, was proposed by Ebrahim Mamdani and is widely used for control systems. It's known for its intuitive approach because its output is a fuzzy set that requires defuzzification.

The *Sugeno FIS*, also known as the Takagi-Sugeno-Kang (TSK) method, was introduced by Michio Sugeno and differs from Mamdani by producing a crisp output directly, without a separate defuzzification step.

Mamdani Fuzzy Inference System



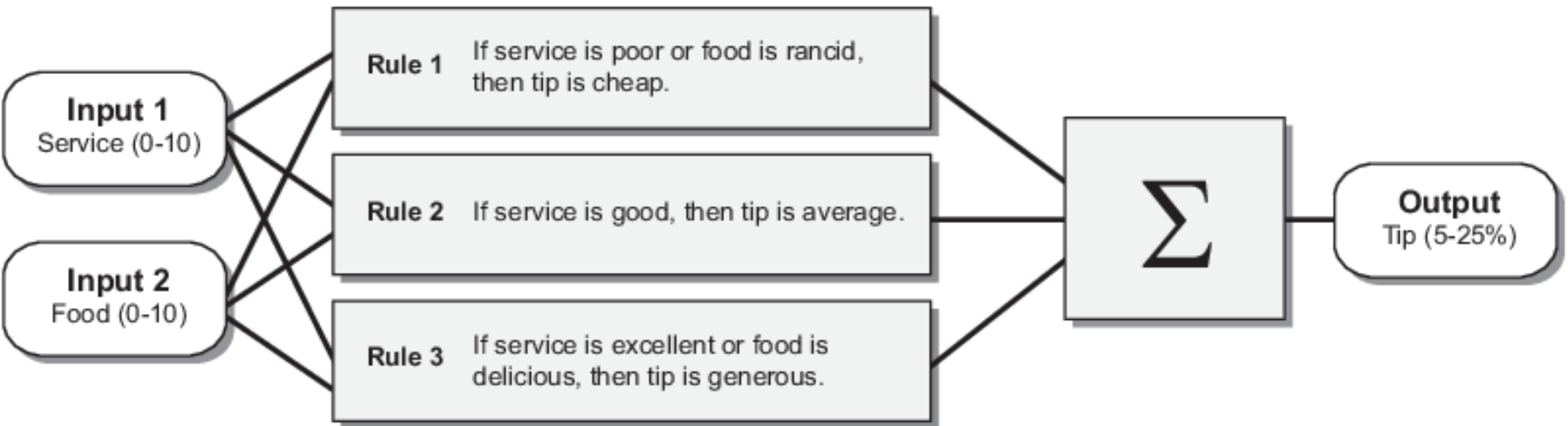
Fuzzy Inference System

The fuzzy inference system for the two-input, one-output, and three-rule tipping problem. In this problem we take *service* and *food quality* as inputs and computes a tip percentage using the following rules:

- 1.If the service is poor or the food is rancid, then tip is cheap.
- 2.If the service is good, then tip is average.
- 3.If the service is excellent or the food is delicious, then tip is generous.

Fuzzy Inference System

Dinner for Two
a 2 input, 1 output, 3 rule system



The inputs are crisp (non-fuzzy) numbers limited to a specific range.



All rules are evaluated in parallel using fuzzy reasoning.



The results of the rules are combined and distilled (defuzzified).

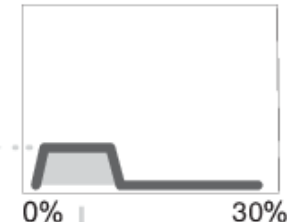
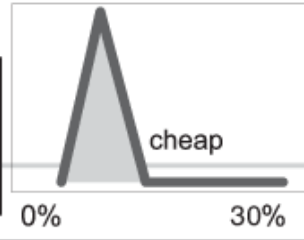
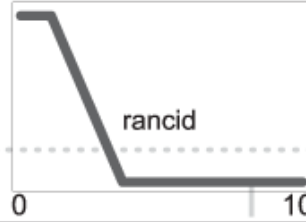
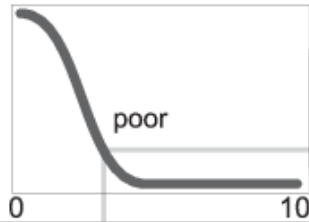
The result is a crisp (non-fuzzy) number.

1. Fuzzify inputs.

2. Apply fuzzy operation (OR = max).

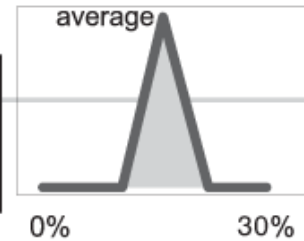
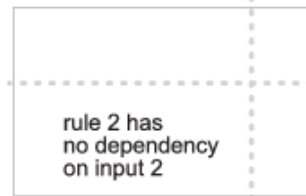
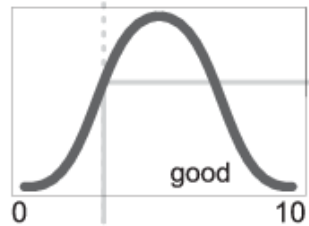
3. Apply implication method (min).

1.



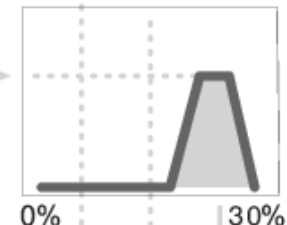
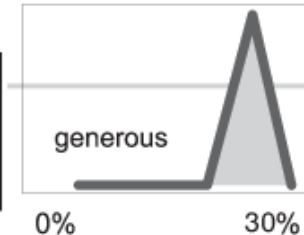
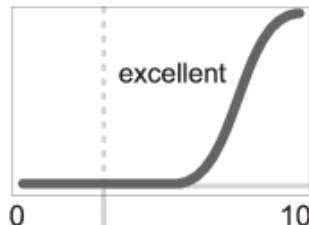
If **service is poor** or **food is rancid** then **tip = cheap**

2.



If **service is good** then **tip = average**

3.



If **service is excellent** or **food is delicious** then **tip = generous**

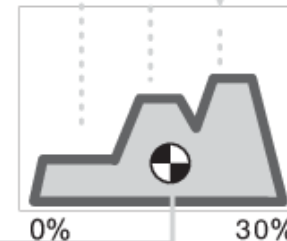
service = 3

input 1

food = 8

input 2

4. Apply aggregation method (max).



5. Defuzzify (centroid).

tip = 16.7%

output

Fuzzy Inference System

Problem: To implement a **Mamdani-type fuzzy logic comfort controller** that estimates the comfort level of a human occupant based on temperature and humidity inputs **without using any built-in fuzzy toolbox functions**. The controller must define membership functions manually, construct a fuzzy rule base, perform fuzzification, inference, aggregation, and defuzzification, and finally provide a **crisp comfort percentage** output.

The system should incorporate:

1. Temperature as an input variable in the range of 0–50°C, classified as Low, Normal, and High via Gaussian and triangular membership functions.
2. Humidity as an input variable in the range of 0–100%, categorized as Low, Normal, and High using trapezoidal and Gaussian membership functions.
3. Comfort level as the output variable between 0–100%, represented by Poor, Normal, and Excellent membership functions.

Implementation

```
import numpy as np  
import matplotlib.pyplot as plt
```

Implementation

```
# -----  
# Membership Functions  
# -----  
  
def step(x):  
    return np.where(x >= 0, 1, 0)  
  
def gaussf(x, c, v):  
    return np.exp(-((x - c) ** 2) / (2 * v ** 2))
```

Implementation

```
def trif(x, a, b, c):  
    s = 1 / (b - a)  
    return (  
        s * (x - a) * step(x - a)  
        - 2 * s * (x - b) * step(x - b)  
        + s * (x - c) * step(x - c)  
    )
```

```
def trapf(x, a, b, c, d):  
    s1 = 1 / (b - a)  
    s2 = 1 / (d - c)  
    return (  
        s1 * (x - a) * step(x - a)  
        - s1 * (x - b) * step(x - b)  
        - s2 * (x - c) * step(x - c)  
        + s2 * (x - d) * step(x - d)  
    )
```


Implementation

```
# Universe of discourse
x1 = np.linspace(0, 50, 5000)    # Temperature
x2 = np.linspace(0, 100, 10000)  # Humidity
o1 = np.linspace(0, 100, 10000) # Output: Comfort level

# Temperature membership functions
x11 = gaussf(x1, 0, 7)           # Low Temp
x12 = trif(x1, 10, 25, 40)       # Normal Temp
x13 = gaussf(x1, 50, 7)          # High Temp

# Humidity membership functions
x21 = trapf(x2, -1, 0, 10, 37)   # Low Humidity
x22 = gaussf(x2, 50, 10)         # Normal Humidity
x23 = trapf(x2, 62, 90, 100, 101) # High Humidity

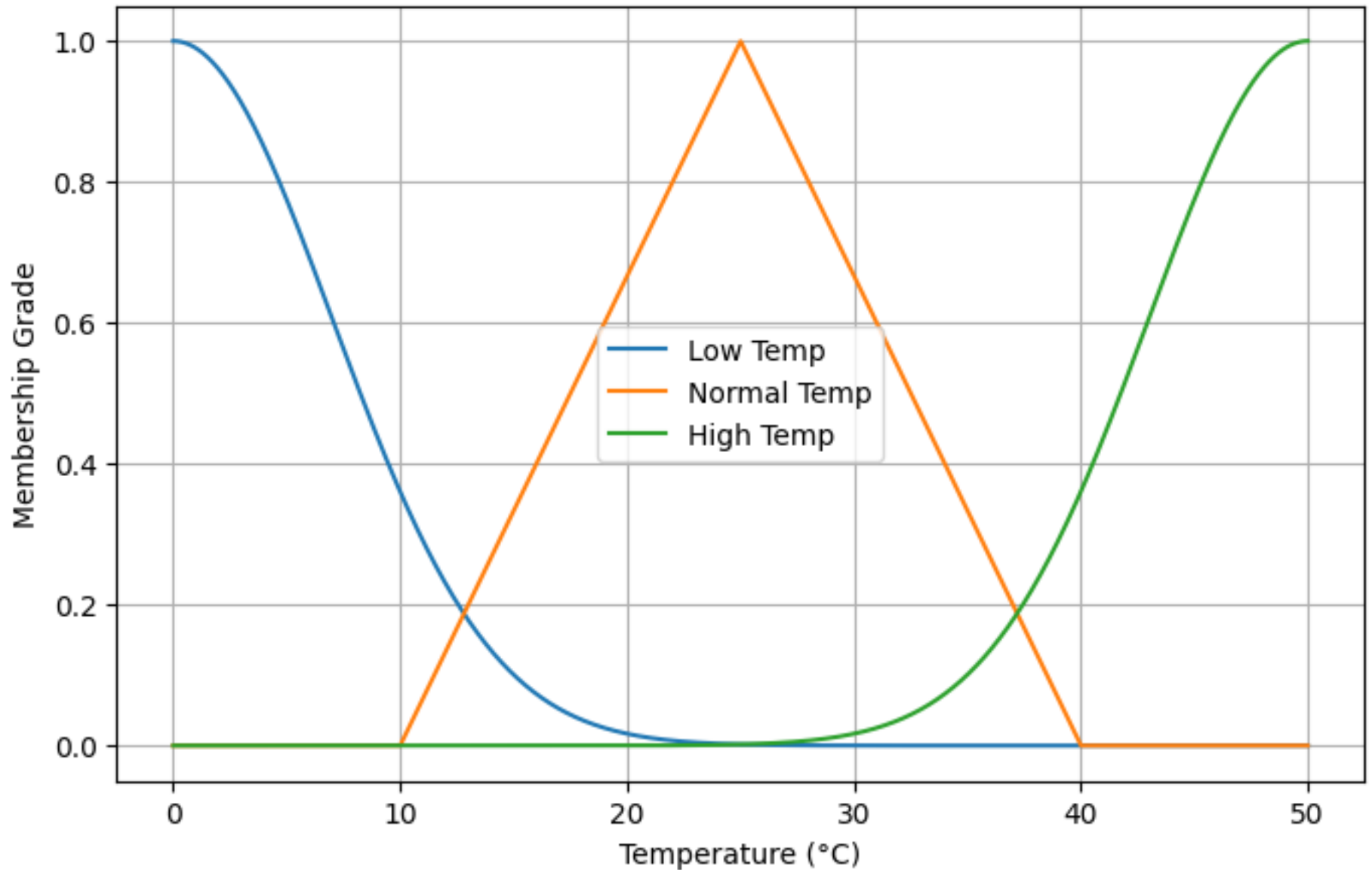
# Output Comfort membership functions
o11 = trapf(o1, -1, 0, 25, 50)    # Poor Comfort
o12 = trif(o1, 25, 50, 75)        # Normal Comfort
o13 = trapf(o1, 50, 75, 100, 101) # Excellent Comfort
```

Implementation

```
# -----  
# Plot Temperature Membership Functions  
# -----  
plt.figure(figsize=(8,5))  
plt.plot(x1, x11, label='Low Temp')  
plt.plot(x1, x12, label='Normal Temp')  
plt.plot(x1, x13, label='High Temp')  
plt.title("Temperature Membership Functions")  
plt.xlabel("Temperature (°C)")  
plt.ylabel("Membership Grade")  
plt.grid()  
plt.legend()  
plt.show()
```

Implementation

Temperature Membership Functions

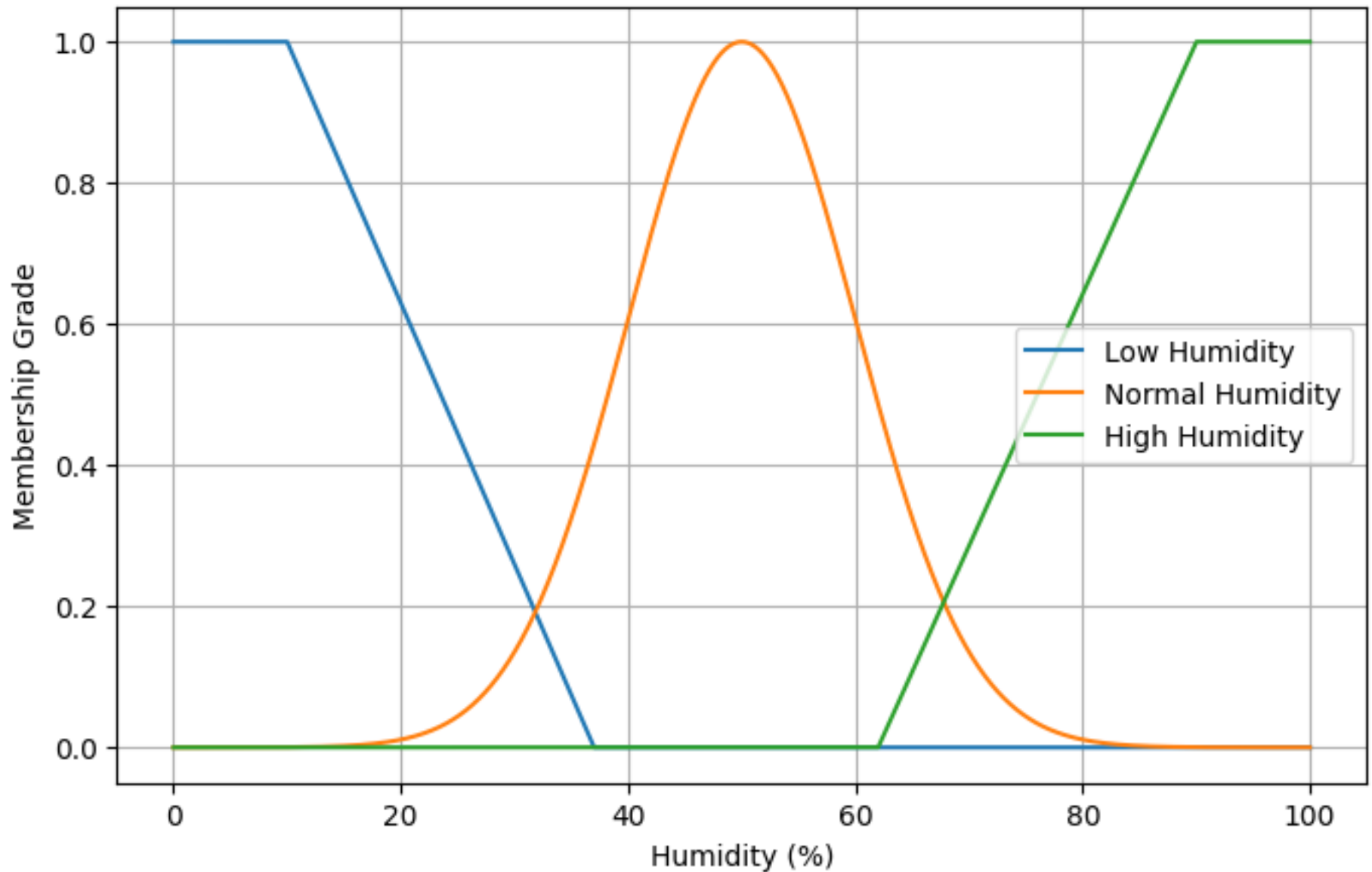


Implementation

```
# -----  
# Plot Humidity Membership Functions  
# -----  
plt.figure(figsize=(8,5))  
plt.plot(x2, x21, label='Low Humidity')  
plt.plot(x2, x22, label='Normal Humidity')  
plt.plot(x2, x23, label='High Humidity')  
plt.title("Humidity Membership Functions")  
plt.xlabel("Humidity (%)")  
plt.ylabel("Membership Grade")  
plt.grid()  
plt.legend()  
plt.show()
```

Implementation

Humidity Membership Functions

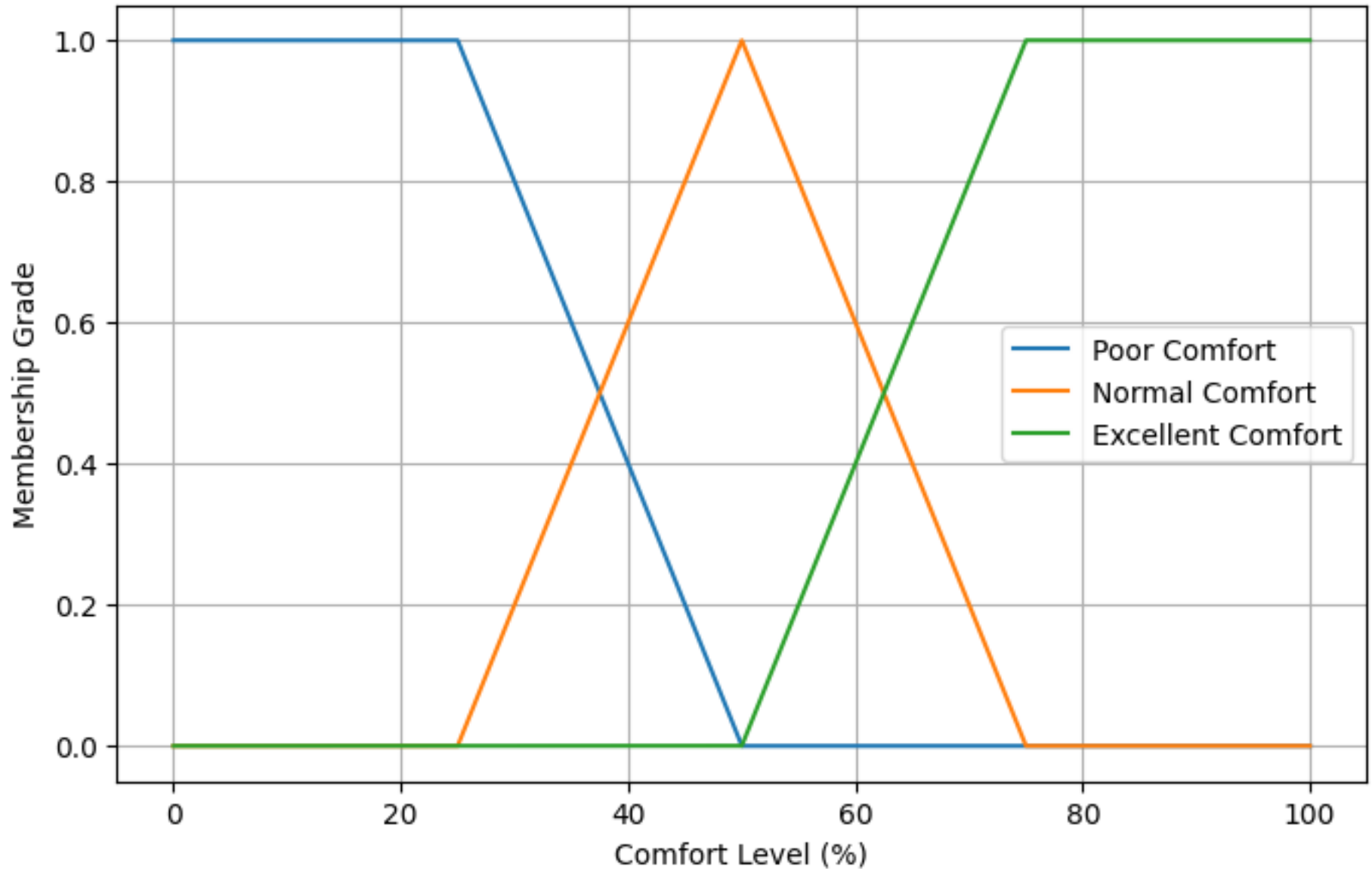


Implementation

```
# -----  
# Plot Output Comfort Membership Functions  
# -----  
plt.figure(figsize=(8,5))  
plt.plot(o1, o11, label='Poor Comfort')  
plt.plot(o1, o12, label='Normal Comfort')  
plt.plot(o1, o13, label='Excellent Comfort')  
plt.title("Output Comfort Level Membership Functions")  
plt.xlabel("Comfort Level (%)")  
plt.ylabel("Membership Grade")  
plt.grid()  
plt.legend()  
plt.show()
```

Implementation

Output Comfort Level Membership Functions



Implementation

Fuzzy Rule Base (Comfort)

		Humidity		
		Low	Normal	High
Temperature	Low	Excellent	Normal	Poor
	Normal	Excellent	Normal	Poor
	High	Normal	Poor	Poor

Output: Comfort

Implementation

```
# -----  
# Provide inputs  
# -----
```

```
temp = 18  
hum = 38
```

```
# Fuzzification
```

```
m11 = np.interp(temp, x1, x11)  
m12 = np.interp(temp, x1, x12)  
m13 = np.interp(temp, x1, x13)
```

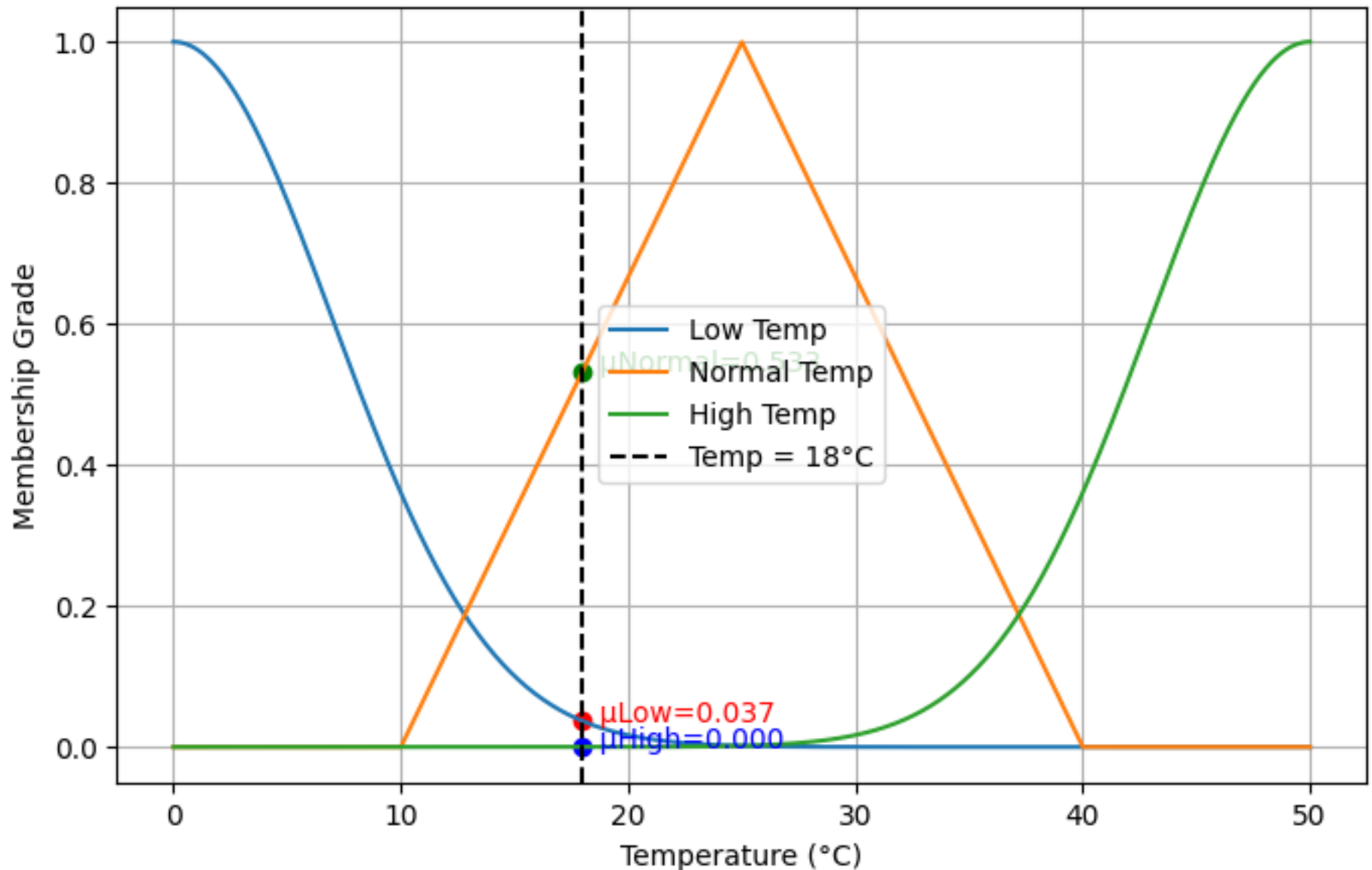
```
m21 = np.interp(hum, x2, x21)  
m22 = np.interp(hum, x2, x22)  
m23 = np.interp(hum, x2, x23)
```

Implementation

```
# -----  
# Show Temperature = 18°C on Temperature MF  
# -----  
  
plt.figure(figsize=(8,5))  
plt.plot(x1, x11, label='Low Temp')  
plt.plot(x1, x12, label='Normal Temp')  
plt.plot(x1, x13, label='High Temp')  
  
# Vertical line  
plt.axvline(temp, color='k', linestyle='--', label=f'Temp = {temp}°C')  
  
# Points at membership values  
plt.scatter([temp], [m11], color='red')  
plt.scatter([temp], [m12], color='green')  
plt.scatter([temp], [m13], color='blue')  
  
# Labels  
plt.text(temp, m11, f'  μLow={m11:.3f}', color='red')  
plt.text(temp, m12, f'  μNormal={m12:.3f}', color='green')  
plt.text(temp, m13, f'  μHigh={m13:.3f}', color='blue')  
  
plt.title("Temperature Membership Functions with Input = 18°C")  
plt.xlabel("Temperature (°C)")  
plt.ylabel("Membership Grade")  
plt.grid()  
plt.legend()  
plt.show()
```

Implementation

Temperature Membership Functions with Input = 18°C

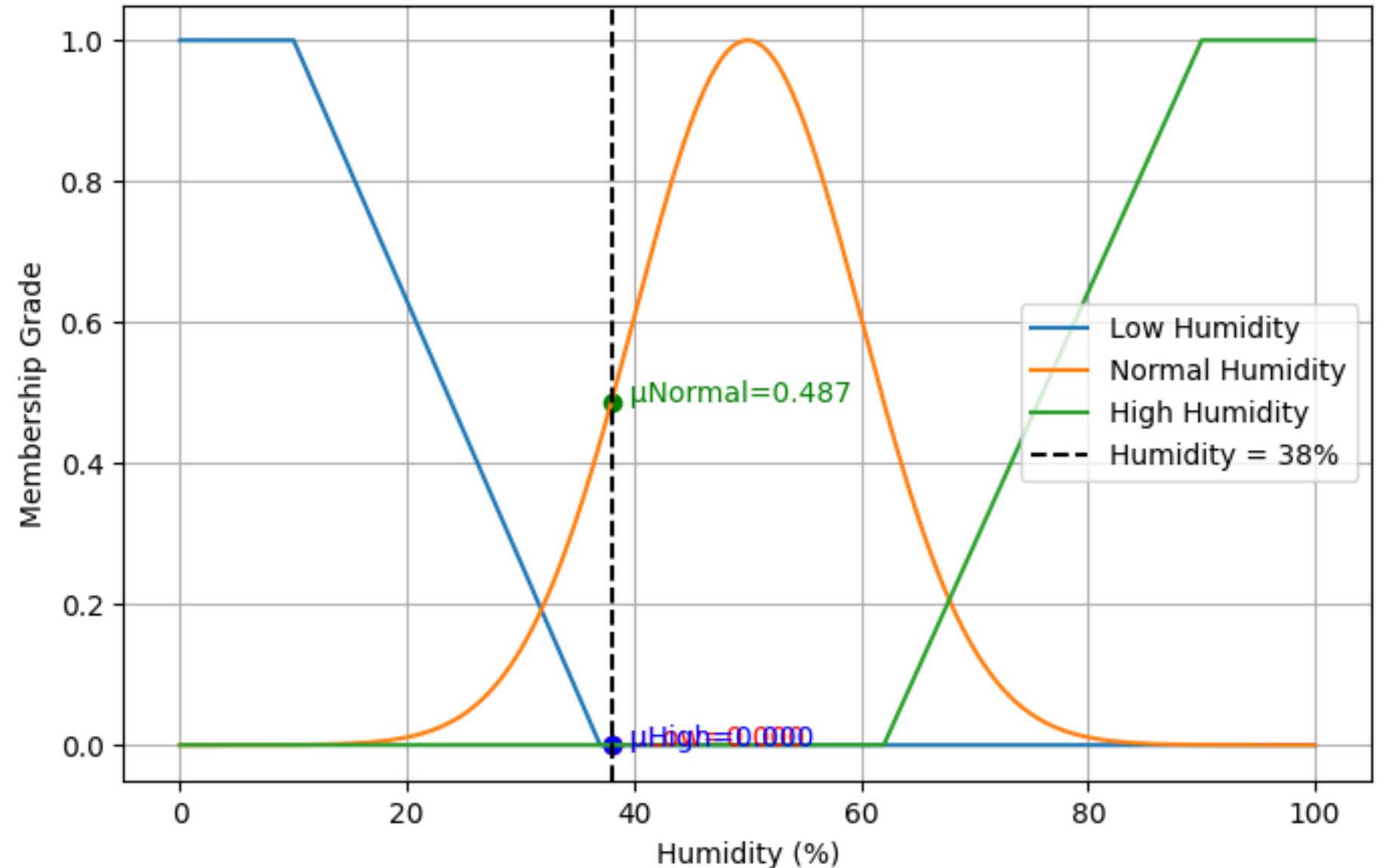


Implementation

```
# -----  
# Show Humidity = 38% on Humidity MF  
# -----  
  
plt.figure(figsize=(8,5))  
plt.plot(x2, x21, label='Low Humidity')  
plt.plot(x2, x22, label='Normal Humidity')  
plt.plot(x2, x23, label='High Humidity')  
  
# Vertical line  
plt.axvline(hum, color='k', linestyle='--', label=f'Humidity = {hum}%')  
  
# Points at membership values  
plt.scatter([hum], [m21], color='red')  
plt.scatter([hum], [m22], color='green')  
plt.scatter([hum], [m23], color='blue')  
  
# Labels  
plt.text(hum, m21, f'  $\mu_{\text{Low}}=\{m21:.3f\}$ ', color='red')  
plt.text(hum, m22, f'  $\mu_{\text{Normal}}=\{m22:.3f\}$ ', color='green')  
plt.text(hum, m23, f'  $\mu_{\text{High}}=\{m23:.3f\}$ ', color='blue')  
  
plt.title("Humidity Membership Functions with Input = 38%")  
plt.xlabel("Humidity (%)")  
plt.ylabel("Membership Grade")  
plt.grid()  
plt.legend()  
plt.show()
```

Implementation

Humidity Membership Functions with Input = 38%



Implementation

Rule base (Mamdani MIN operator)

```
d1 = np.minimum(o13, np.minimum(m11, m21))  
d2 = np.minimum(o12, np.minimum(m11, m22))  
d3 = np.minimum(o11, np.minimum(m11, m23))
```

```
d4 = np.minimum(o13, np.minimum(m12, m21))  
d5 = np.minimum(o12, np.minimum(m12, m22))  
d6 = np.minimum(o11, np.minimum(m12, m23))
```

```
d7 = np.minimum(o12, np.minimum(m13, m21))  
d8 = np.minimum(o11, np.minimum(m13, m22))  
d9 = np.minimum(o11, np.minimum(m13, m23))
```

Implementation

Rule	Code Expression	Linguistic Form
d1	<code>min(o13, min(m11, m21))</code>	IF Temp = Low AND Hum = Low → Comfort = Excellent
d2	<code>min(o12, min(m11, m22))</code>	IF Temp = Low AND Hum = Normal → Comfort = Normal
d3	<code>min(o11, min(m11, m23))</code>	IF Temp = Low AND Hum = High → Comfort = Poor
d4	<code>min(o13, min(m12, m21))</code>	IF Temp = Normal AND Hum = Low → Comfort = Excellent
d5	<code>min(o12, min(m12, m22))</code>	IF Temp = Normal AND Hum = Normal → Comfort = Normal
d6	<code>min(o11, min(m12, m23))</code>	IF Temp = Normal AND Hum = High → Comfort = Poor
d7	<code>min(o12, min(m13, m21))</code>	IF Temp = High AND Hum = Low → Comfort = Normal
d8	<code>min(o11, min(m13, m22))</code>	IF Temp = High AND Hum = Normal → Comfort = Poor
d9	<code>min(o11, min(m13, m23))</code>	IF Temp = High AND Hum = High → Comfort = Poor

Implementation

```
# Aggregation (MAX)
```

```
d = np.maximum.reduce([d1, d2, d3, d4, d5, d6, d7, d8, d9])
```

```
# Defuzzification (COG)
```

```
crisp_output = np.sum(o1 * d) / np.sum(d)
```

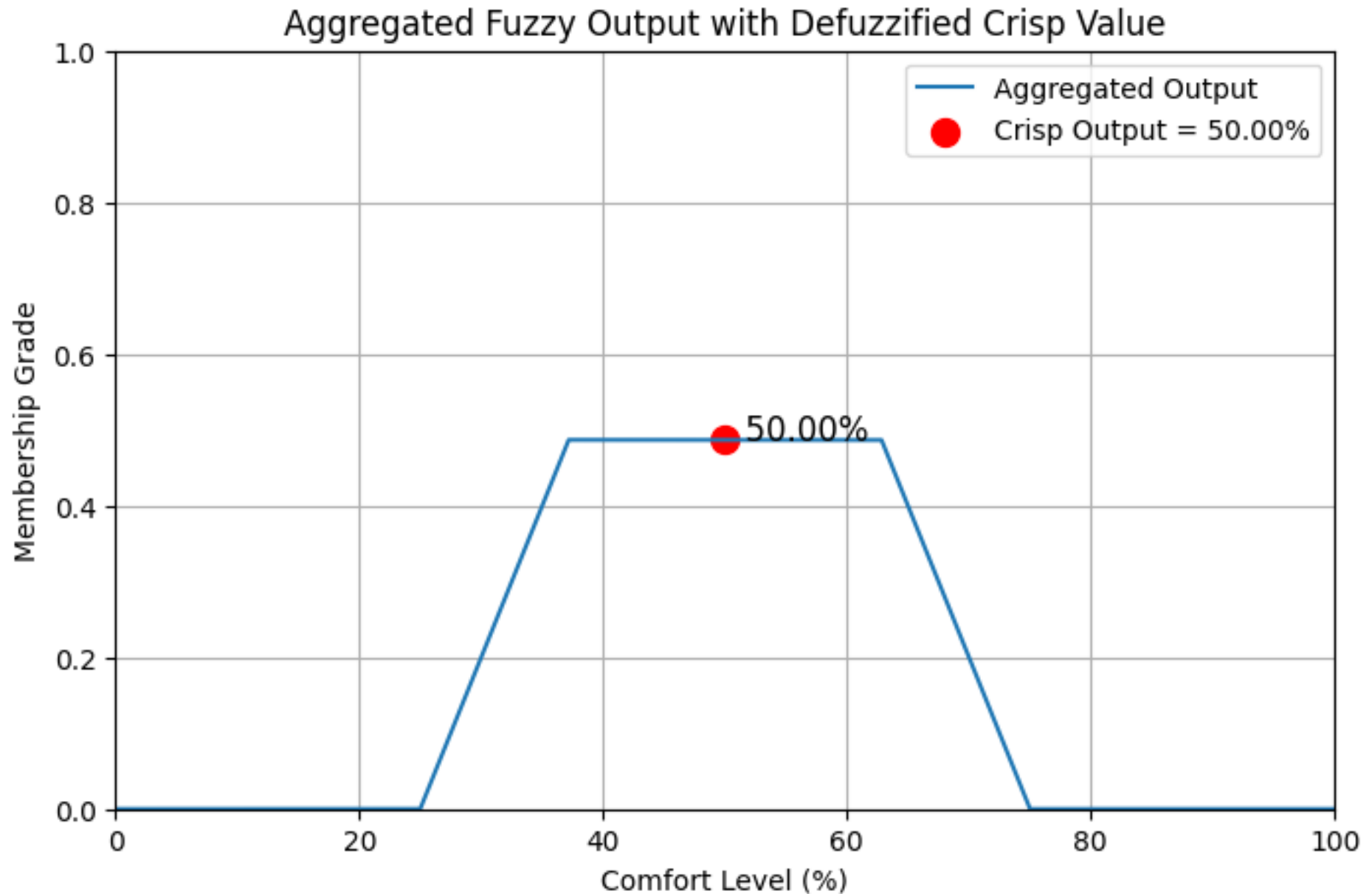
```
print(f"Comfort Level = {crisp_output:.2f} %")
```

Comfort Level = 50.00 %

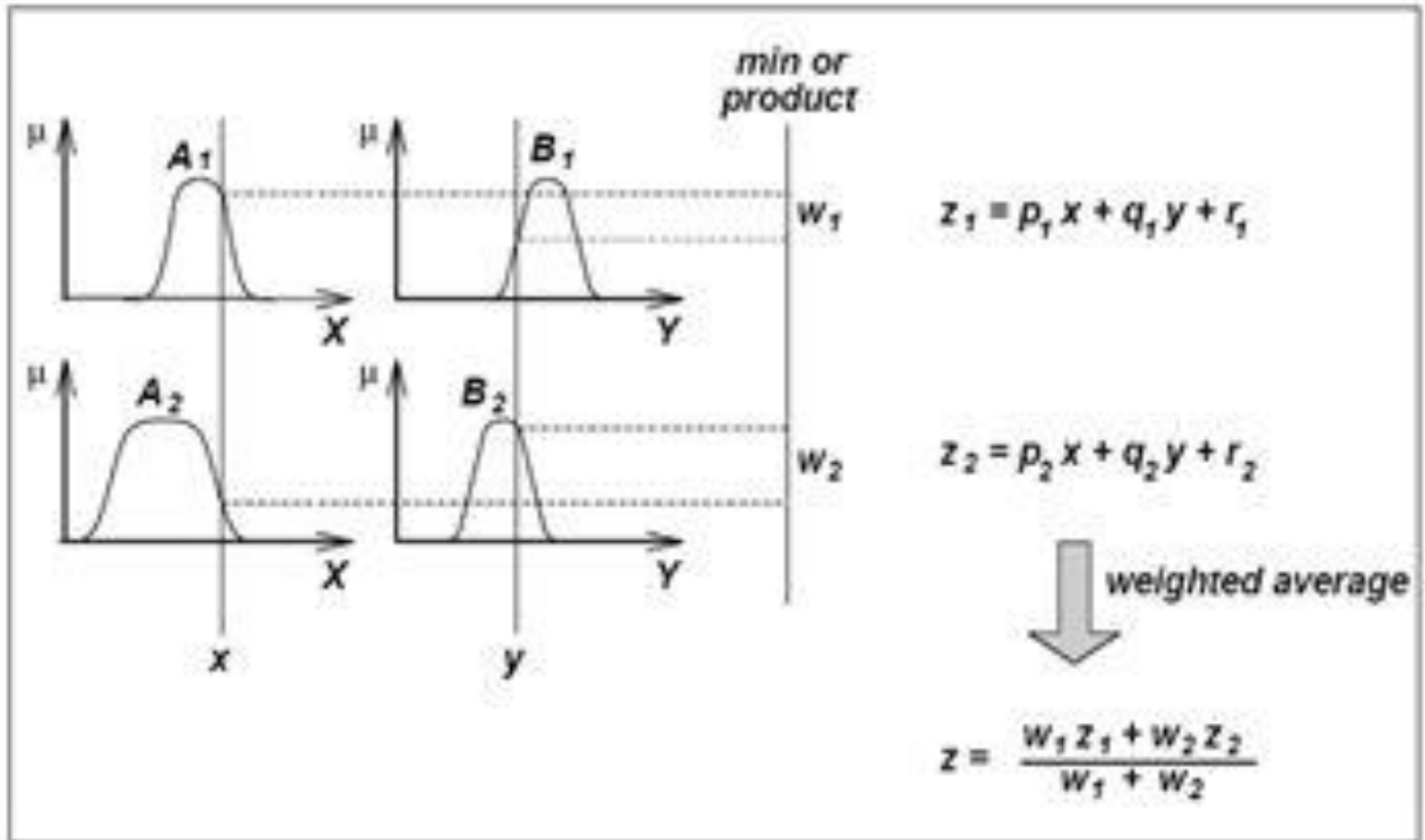
Implementation

```
# -----  
# Plot Aggregated Output with Crisp Point Marked  
# -----  
  
plt.figure(figsize=(8,5))  
plt.plot(o1, d, label="Aggregated Output")  
plt.scatter([crisp_output], [np.interp(crisp_output, o1, d)],  
            color='red', s=100, label=f"Crisp Output = {crisp_output:.2f}%")  
plt.text(crisp_output, np.interp(crisp_output, o1, d),  
         f" {crisp_output:.2f}%", fontsize=12)  
  
plt.xlabel("Comfort Level (%)")  
plt.ylabel("Membership Grade")  
plt.title("Aggregated Fuzzy Output with Defuzzified Crisp Value")  
plt.xlim(0, 100)  
plt.ylim(0, 1)  
plt.grid(True)  
plt.legend()  
plt.show()  
  
print(f"\nDefuzzified Comfort Level = {crisp_output:.2f}%\n")
```

Implementation



Sugeno Fuzzy Inference System



Sugeno Fuzzy Inference System

Fuzzy Rules

In a **Sugeno fuzzy model**, each rule looks like:

Rule i : *If x is A_i and y is B_i , then $z_i = p_i x + q_i y + r_i$*

Where:

- A_i and B_i are **fuzzy sets** for inputs x and y ,
- p_i, q_i, r_i are **coefficients**
(defining a linear or constant output function),
- z_i is the **crisp output** of rule i .

Sugeno Fuzzy Inference System

Rule Activation (Firing Strength)

This “AND” operation can be:

Min: $w_i = \min(\mu_{A_i}(x), \mu_{B_i}(y))$

Product: $w_i = \mu_{A_i}(x) \times \mu_{B_i}(y)$

w_i represents the firing strength or weight of each rule.

Sugeno Fuzzy Inference System

Rule Output

Each rule produces an output z_i as a **linear (or constant) function** of inputs:

$$z_i = p_i x + q_i y + r_i$$

Defuzzification (Weighted Average Method)

$$Z = \frac{w_1 z_1 + w_2 z_2 + \cdots + w_n z_n}{w_1 + w_2 + \cdots + w_n}$$

Crisp output the **weighted average** of all rule outputs.

Sugeno Fuzzy Inference System

Example

Rule 1: If x is *Low* and y is *High*, then $z_1 = 2x + y + 1$

Rule 2: If x is *High* and y is *Low*, then $z_2 = x - 3y + 2$

Suppose:

$$\mu_{A_1}(x) = 0.6, \mu_{B_1}(y) = 0.8$$

$$\mu_{A_2}(x) = 0.4, \mu_{B_2}(y) = 0.5$$

Then:

$$w_1 = 0.6 \times 0.8 = 0.48, w_2 = 0.4 \times 0.5 = 0.20$$

Sugeno Fuzzy Inference System

Example

Let $x = 1.5, y = 2.0$:

$$z_1 = 2(1.5) + 2 + 1 = 6$$

$$z_2 = 1.5 - 3(2.0) + 2 = -2.5$$

$$z = \frac{0.48(6) + 0.20(-2.5)}{0.48 + 0.20} = \frac{2.88 - 0.5}{0.68} = 3.50$$

Defuzzified Crisp Output: $z = 3.5$

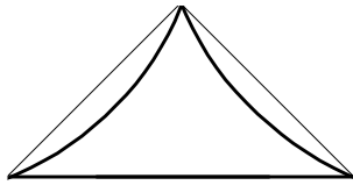
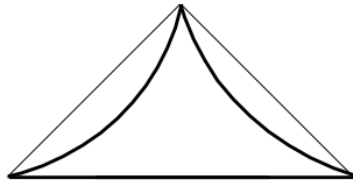
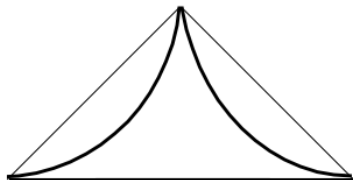
Fuzzy Inference System

Fuzzy Inference System	Advantages
Mamdani	●Intuitive ●Well-suited to human inputs ●More interpretable and rule-based ●Has widespread acceptance
Sugeno	●Computationally efficient ●linear techniques, like PID control ●Functions with optimization ●Guarantees output surface continuity ●Well-suited to mathematical analysis

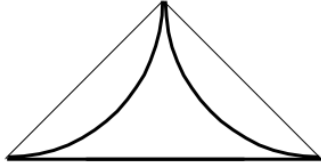
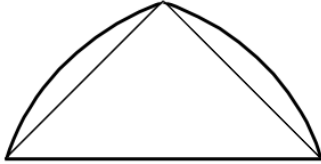
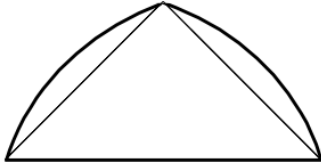
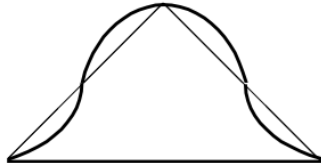
Linguistic Variables and Hedges

- A linguistic variable is a fuzzy variable. For example, the statement “John is tall” implies that the linguistic variable John takes the linguistic value tall.
- Hedges are terms that modify the shape of fuzzy sets. They include adverbs such as very, somewhat, quite, more or less and slightly.

Linguistic Variables and Hedges

<i>Hedge</i>	<i>Mathematical Expression</i>	<i>Graphical Representation</i>
A little	$[\mu_A(x)]^{1.3}$	
Slightly	$[\mu_A(x)]^{1.7}$	
Very	$[\mu_A(x)]^2$	
Extremely	$[\mu_A(x)]^3$	

Linguistic Variables and Hedges

<i>Hedge</i>	<i>Mathematical Expression</i>	<i>Graphical Representation</i>
Very very	$[\mu_A(x)]^4$	
More or less	$\sqrt{\mu_A(x)}$	
Somewhat	$\sqrt{\mu_A(x)}$	
Indeed	$2 [\mu_A(x)]^2$ if $0 \leq \mu_A \leq 0.5$ $1 - 2 [1 - \mu_A(x)]^2$ if $0.5 < \mu_A \leq 1$	

Control System

- A control system for a physical system is an arrangement of hardware components designed to alter, to regulate, or to command, through a control action, so that it exhibits certain desired characteristics or behaviour.
- Physical control systems are typically of two types: open-loop control systems, in which the control action is independent of the physical system output, and closed-loop control systems (also known as feedback control systems), in which the control action depends on the physical system output.

Control System

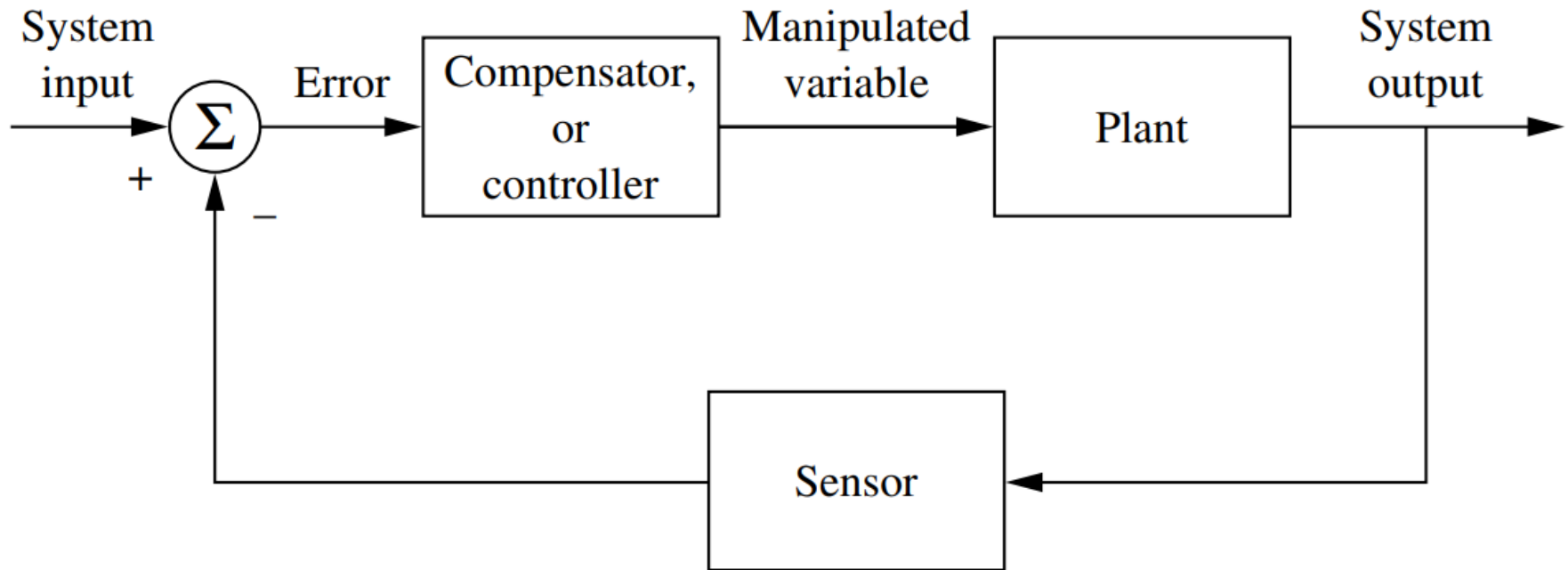


Fig: A Closed-Loop Control System

Fuzzy Control System

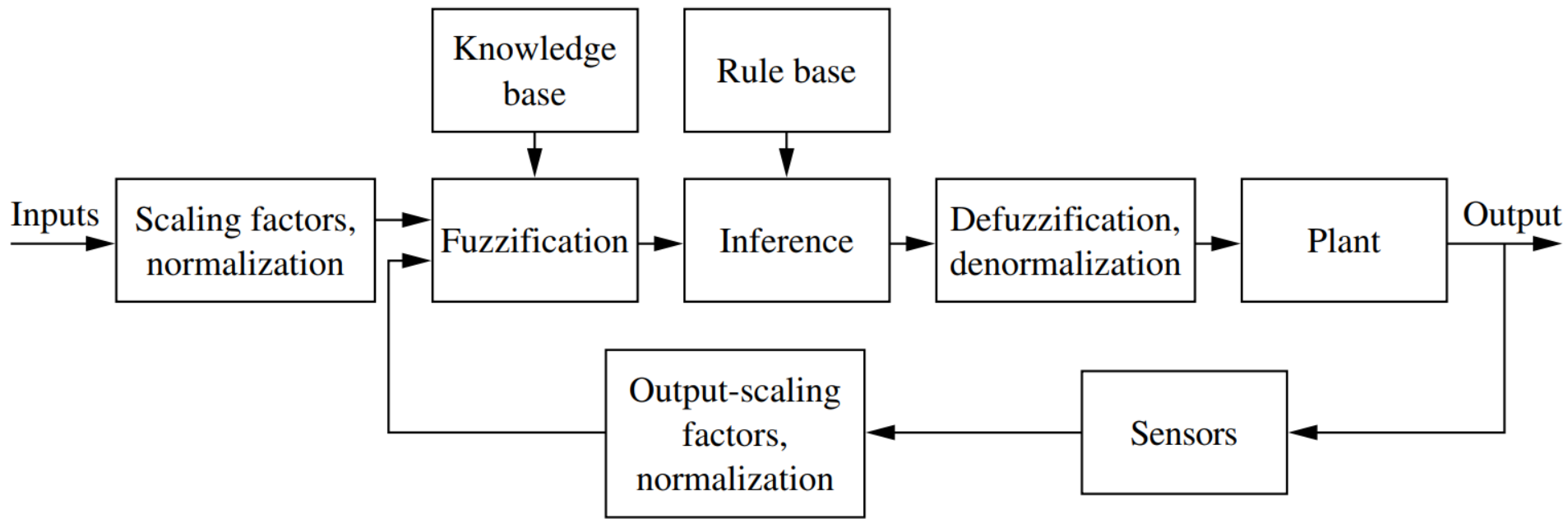


Fig: A simple fuzzy logic control system block diagram

Fuzzy Control System: Example

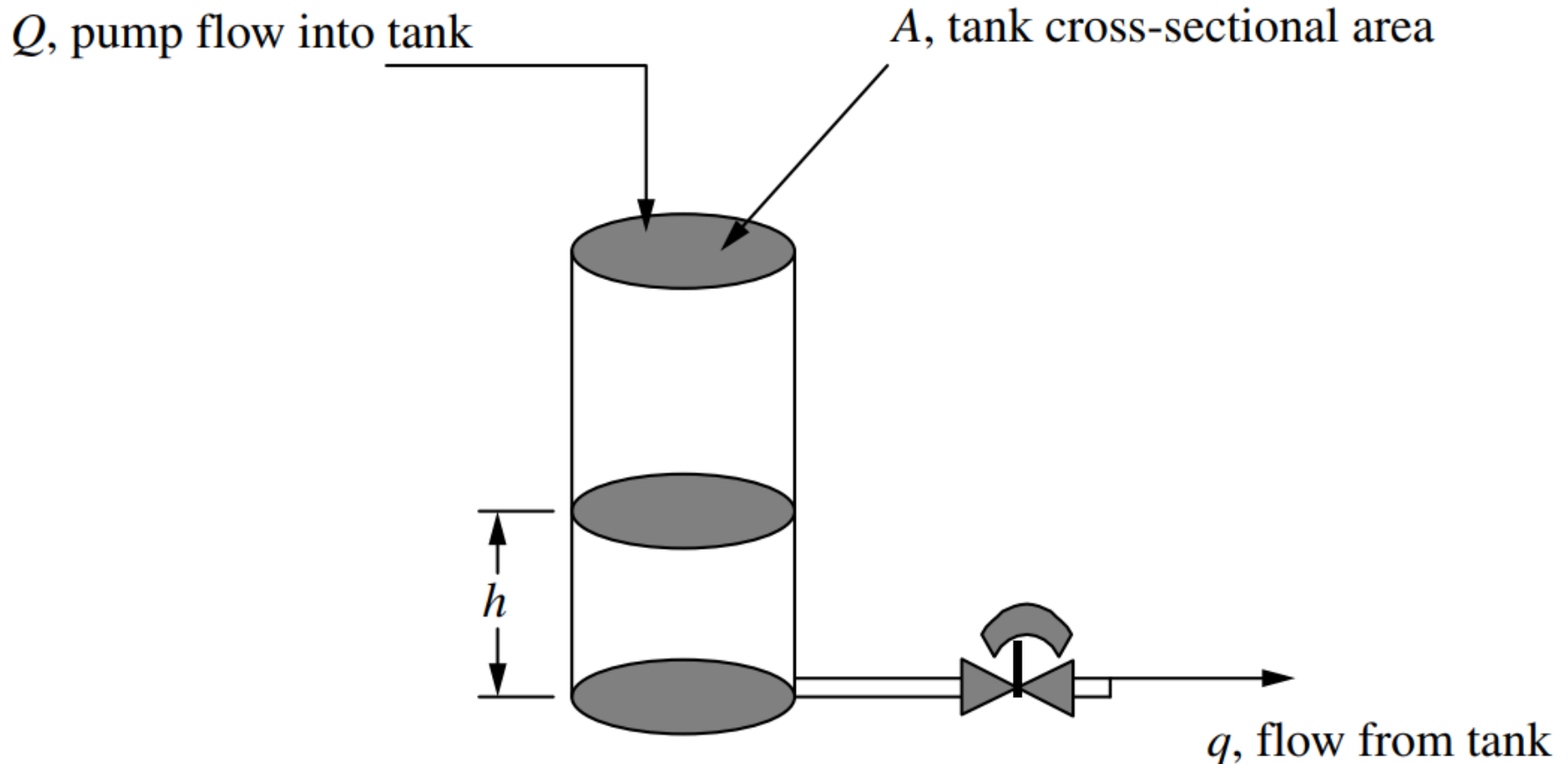


Fig: Tank with a liquid level that needs to be controlled

Fuzzy Control System: Example

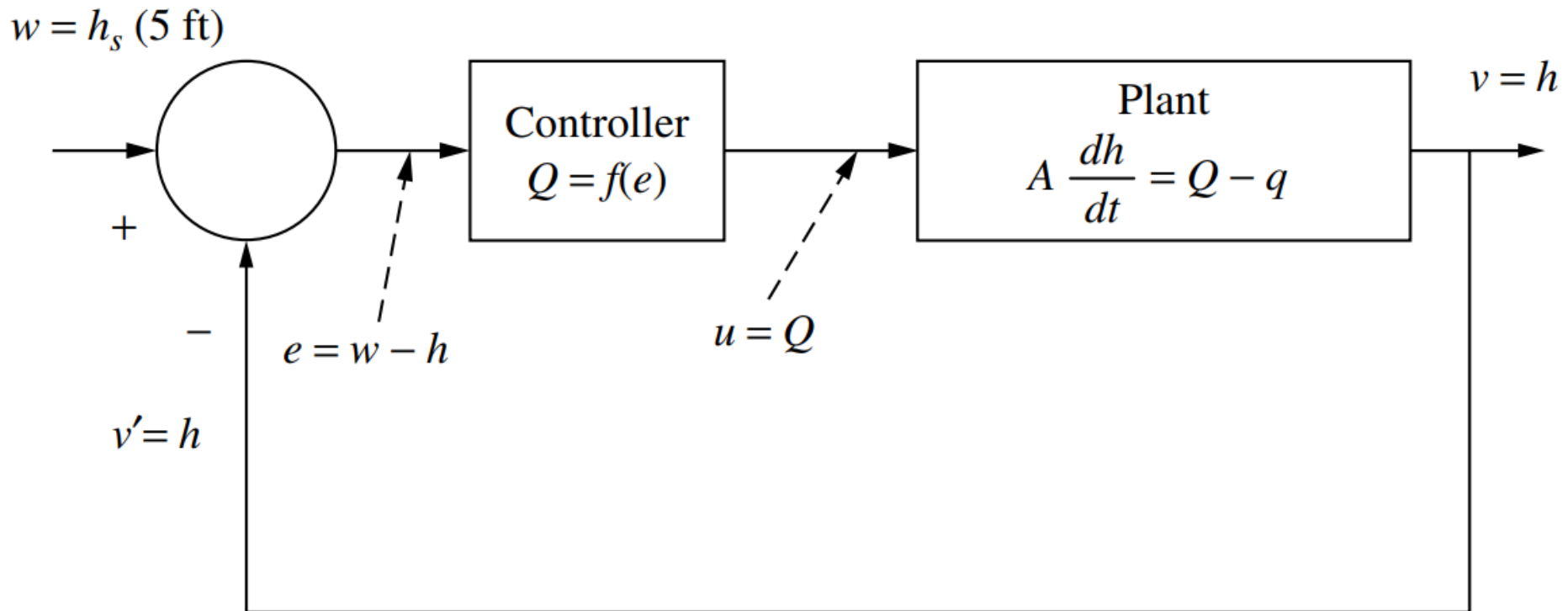


Fig: Closed-loop Liquid level Control System

Fuzzy Control System: Example

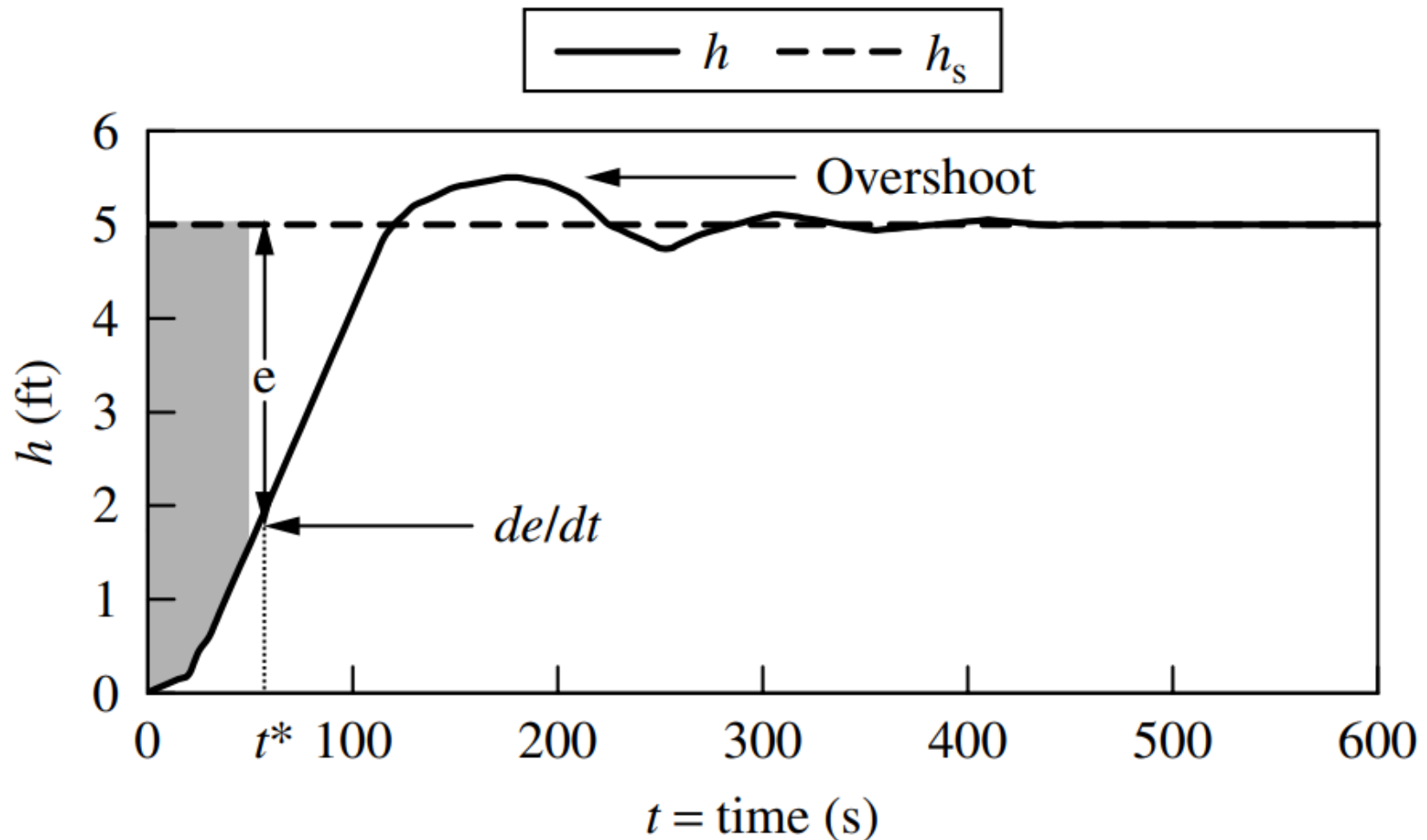


Fig: Time- level response for the Liquid level Control System

Fuzzy Control System: Example

1. If the **Level Error** is **Positive** Then the **Change in Control Action** is **Positive**.
2. If the **Level Error** is **Zero** Then the **Change in Control Action** is **Zero**.
3. If the **Level Error** is **Negative** Then the **Change in Control Action** is **Negative**.

